CI Pipeline with Jenkins (Freestyle, Pipeline, Multibranch)

Overview

This project extends the Jenkins setup to implement a Continuous Integration (CI) pipeline for a Java Maven application. The pipeline automates build, containerization, and image publishing using **Jenkins**.

Technologies:

- Jenkins
- Docker
- Git
- Apache Maven
- Java
- Linux

Pipeline Objective

The CI pipeline performs the following:

1. Connect to Git repository
2. Build Java application (JAR)
3. Build Docker image
4. Push image to private Docker Hub repository / private Nexus repository

Prerequisites

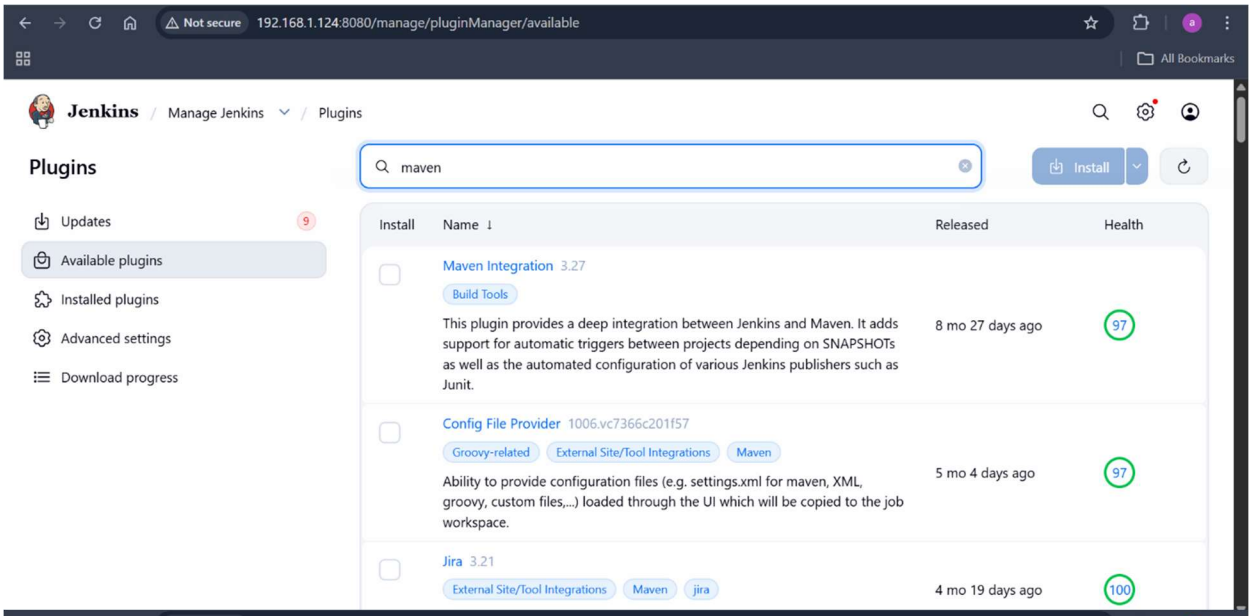
- Jenkins running (Docker container from previous project)
- Docker installed on host
- Git repository with Java Maven project
- Docker Hub account

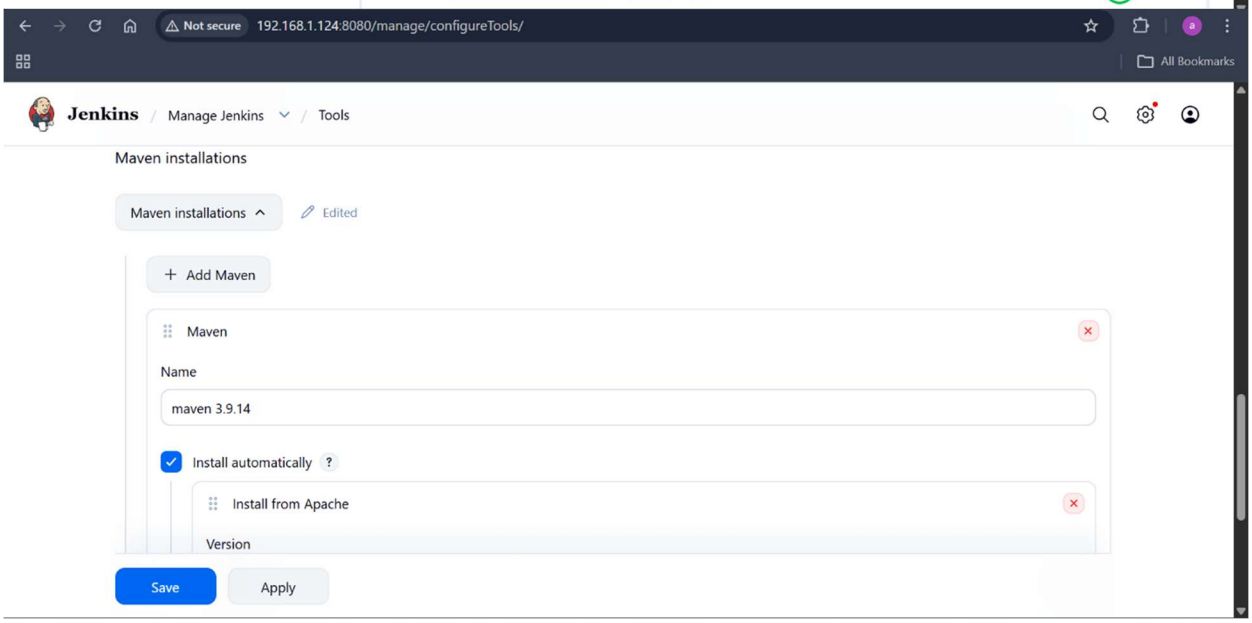
Jenkins Configuration

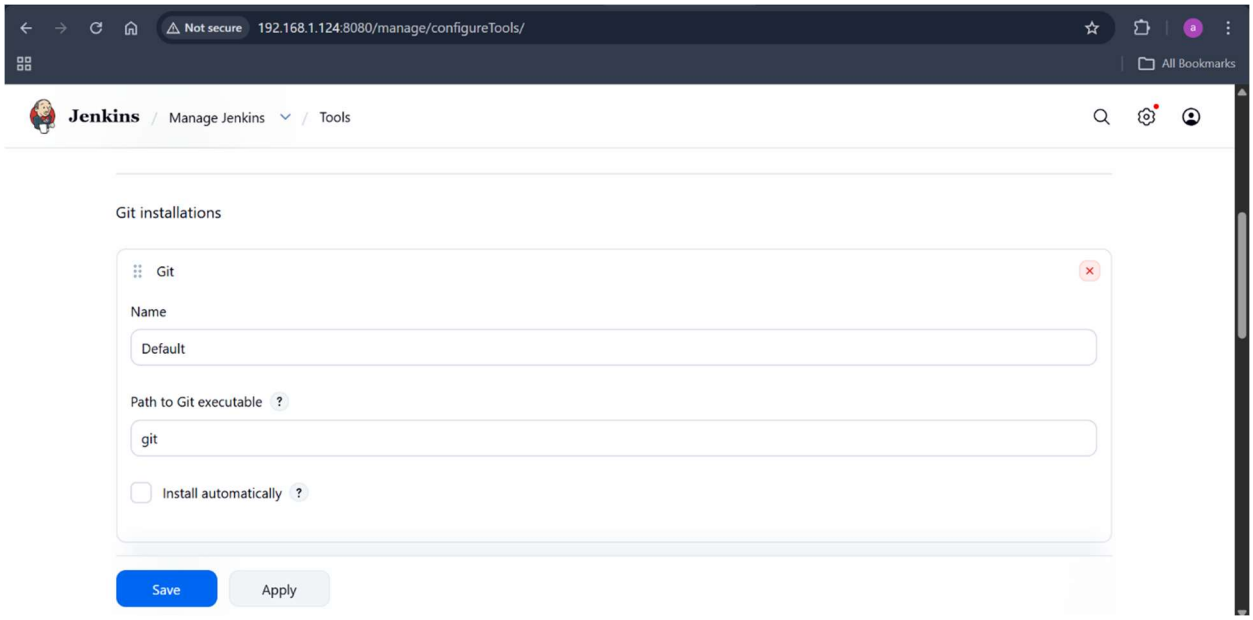
1. Install Required Tools

Maven

- Navigate to: **Manage Jenkins → Plugins → search Maven Integration and install it**
- Navigate to: **Manage Jenkins → Tool**
- Add Maven installation
- Enable automatic installation







Similarly, go to the tool configuration and add Git installations.

Docker

A common way to install docker in a Jenkins container is to mount the docker runtime directory from the host to the running container as a volume.

- Stop the running docker container. It had a volume attached to it for persistent storage.
- Start a new container using two volumes: the old container's volume(to retain data loss) and the docker runtime directory from the host using the following command:

  *docker run -p 8080:8080 -p 5000:5000 -d *

  *-v jenkins_home:/var/jenkins_home *

  -v /var/run/docker.sock:/var/run/docker.sock jenkins/jenkins:lts

- Sign in as a root user in the running container

  docker exec -u 0 -it <container id> bash

- Use a curl command to fetch the latest version of docker and set the appropriate permissions

  curl https://get.docker.com/ > dockerinstall && chmod 777 dockerinstall && ./dockerinstall
- Next we add permissions for the jenkins user in the container to be able to read and write in the docker.sock file.

```
ali-abdul-aleem@ali-abdul-aleem-OptiPlex-3020: ~
root@bbeb07040b25:/# ls -l /var/run/docker.sock
srw-rw-rw- 1 root 125 0 Apr 18 01:13 /var/run/docker.sock
root@bbeb07040b25:/# docker pull redis
Using default tag: latest
latest: Pulling from library/redis
3531af2bc2a9: Pull complete
8f839a2e1cbc: Pull complete
491ff8203928: Pull complete
3f06a39c2815: Pull complete
b017d1abb47c: Pull complete
4f4fb700ef54: Pull complete
ad7bbcf6c7a3: Pull complete
Digest: sha256:25dbb04fc4d6d190eda327dab551631500b0f9ac8f9808e8e63b7fda1ddff196
Status: Downloaded newer image for redis:latest
docker.io/library/redis:latest
root@bbeb07040b25:/#
```

2. Configure Credentials

Navigate to:

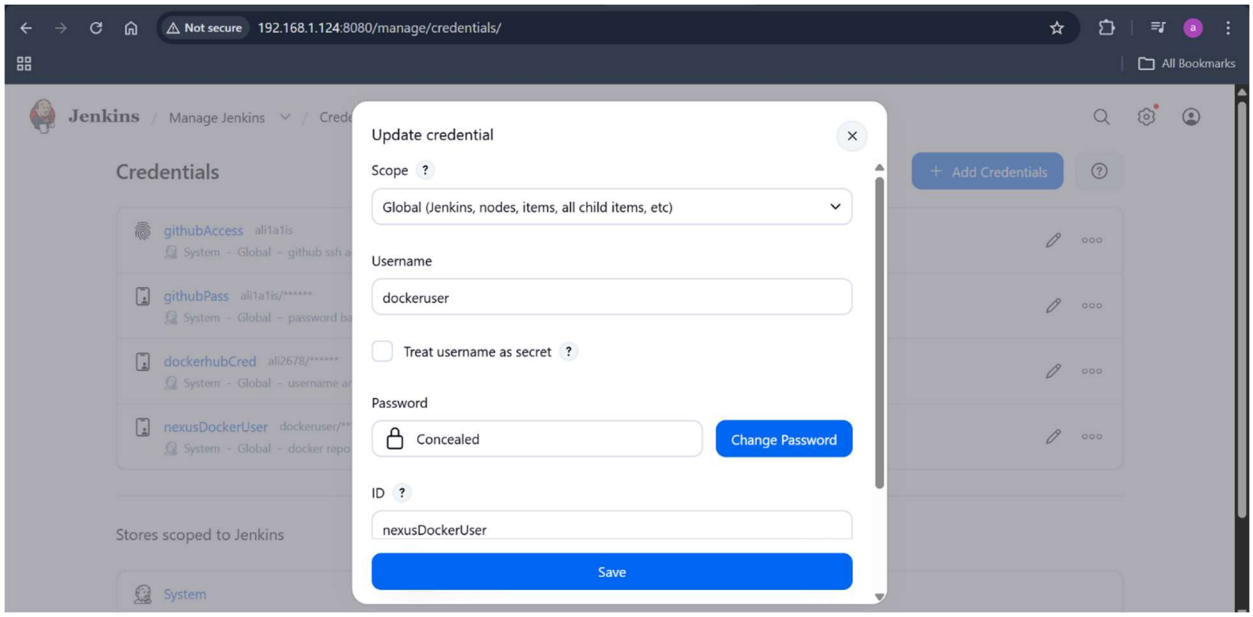
- **Manage Jenkins → Credentials**

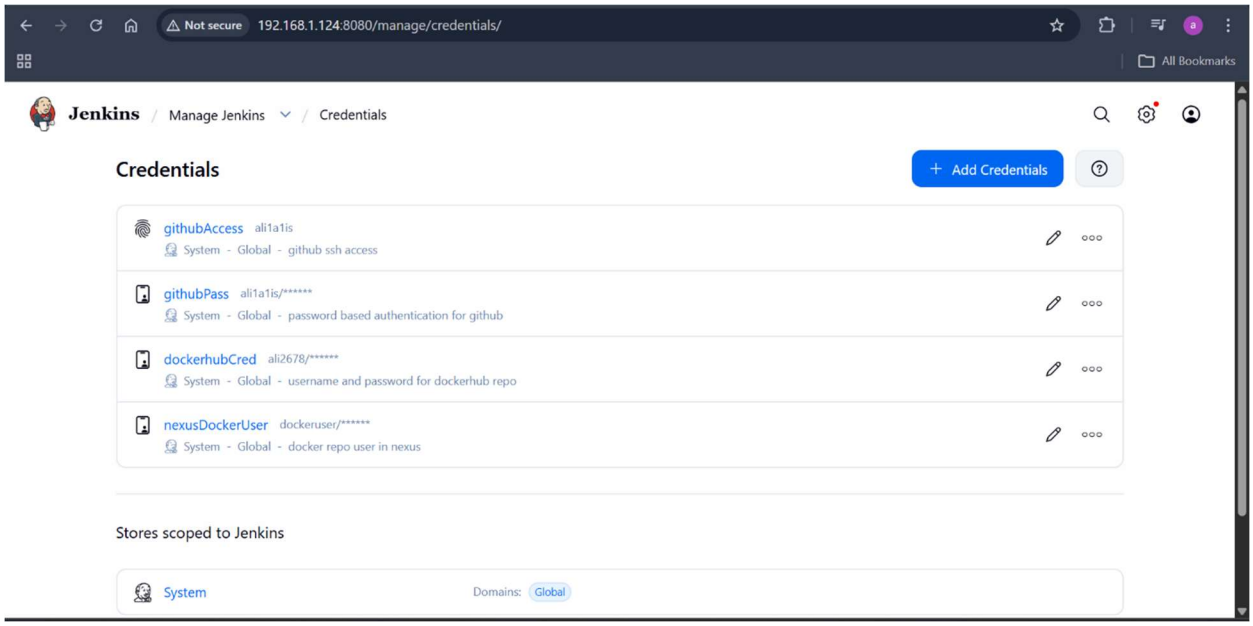
Add:

- Git repository credentials
- Docker Hub credentials
- Nexus private repository credentials

Use:

- Username & Password type





Jenkins Job Types

1. Freestyle Job

Basic job configuration:

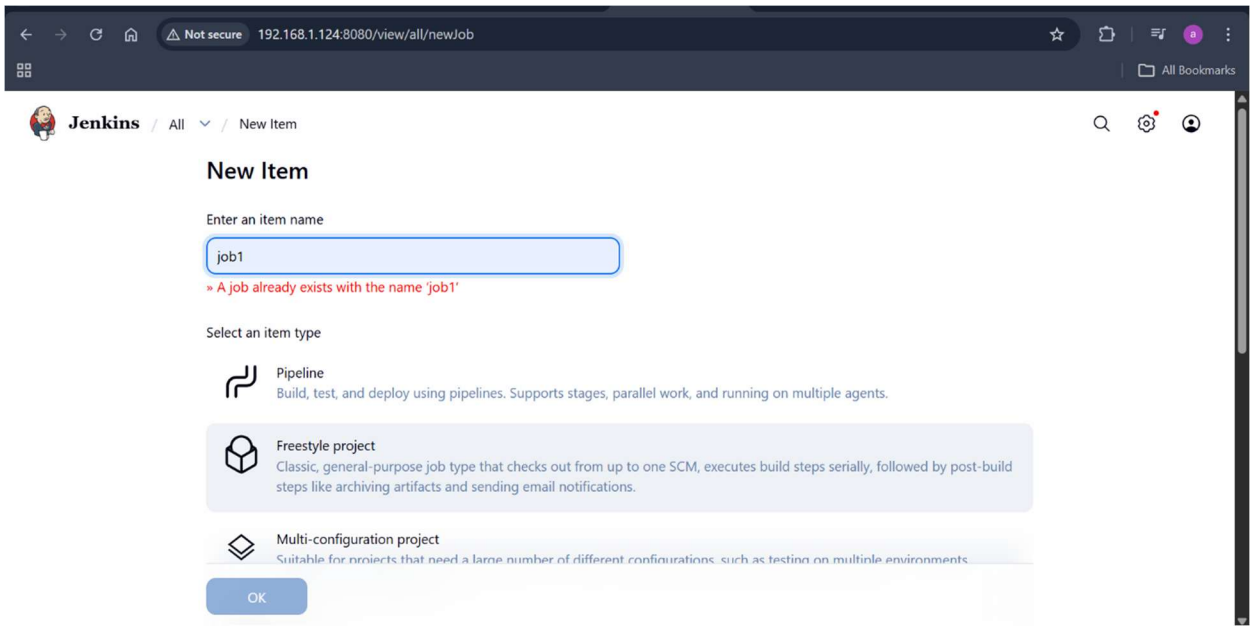
- Source Code Management → Git (repo URL + credentials)

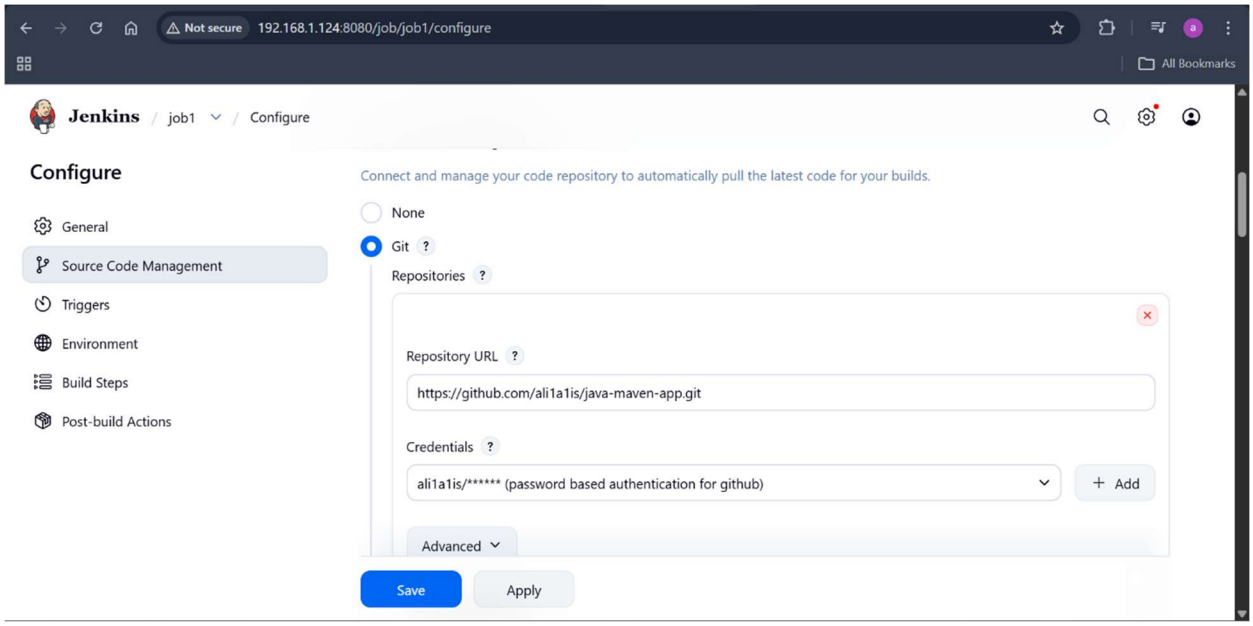
- Environment → Configure environment variables for username and password for nexus docker repository / docker hub repo, depending on where the image is pushed.
- Build Step → Invoke Maven
- Execute docker shell commands to push to repository:

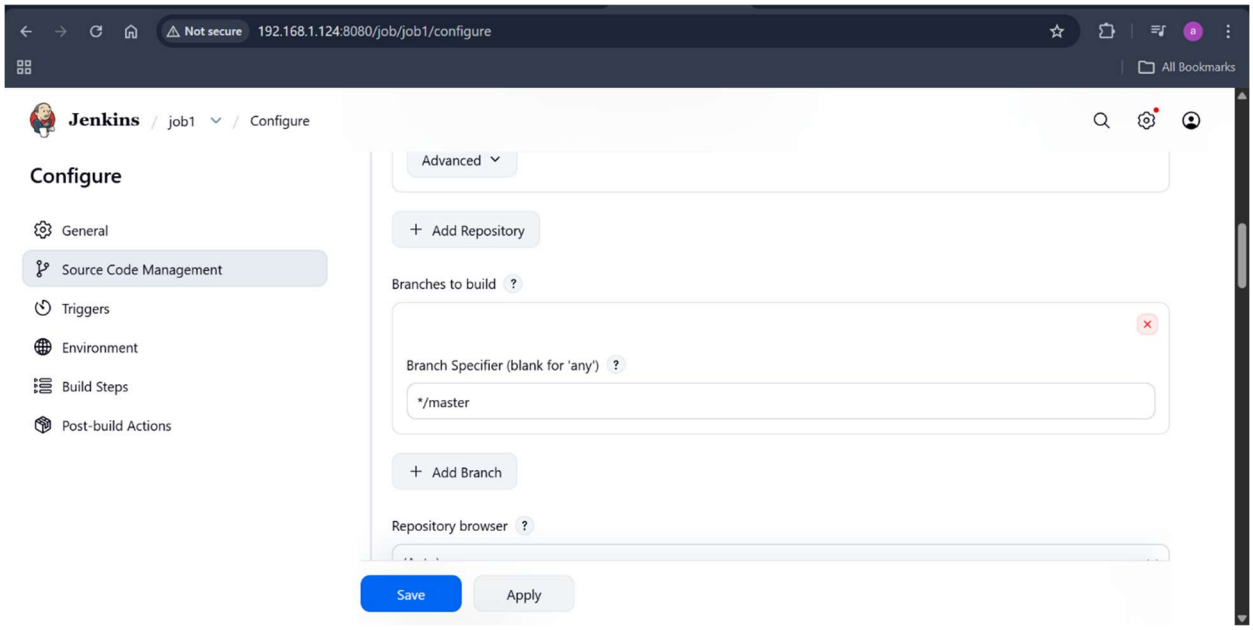
  docker build -t 192.168.1.124:8082/java_mvn_app:1.0 .

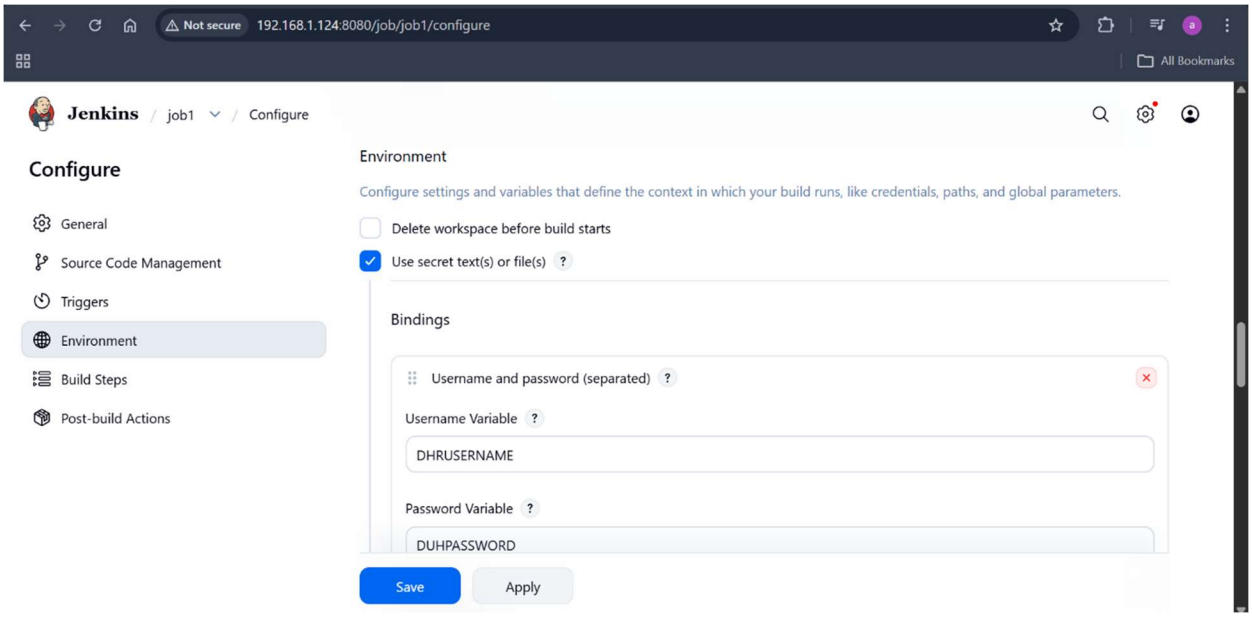
  docker login -u $NDUSERNAME -p $NDPASSWORD 192.168.1.124:8082

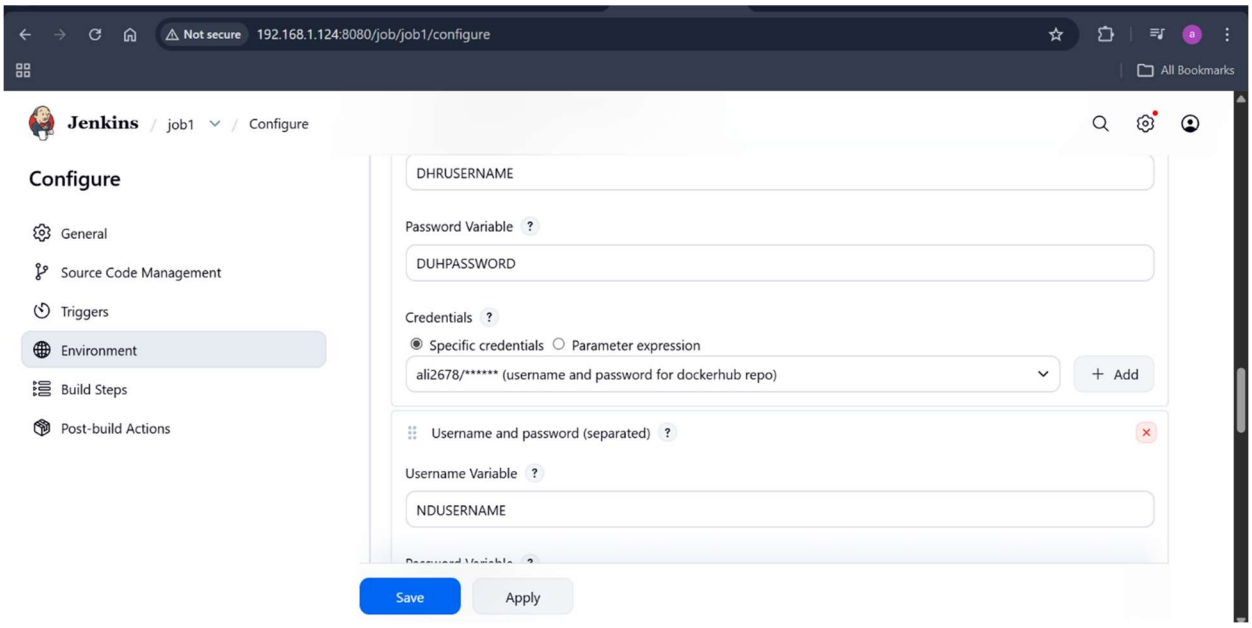
  docker push 192.168.1.124:8082/java_mvn_app:1.0

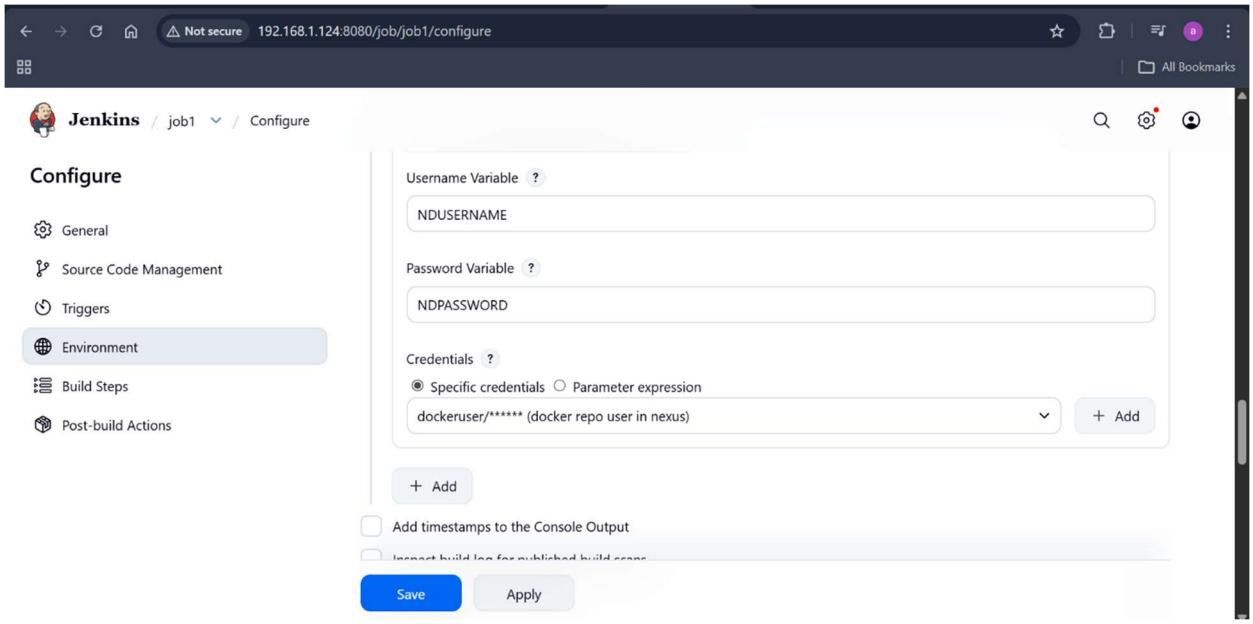


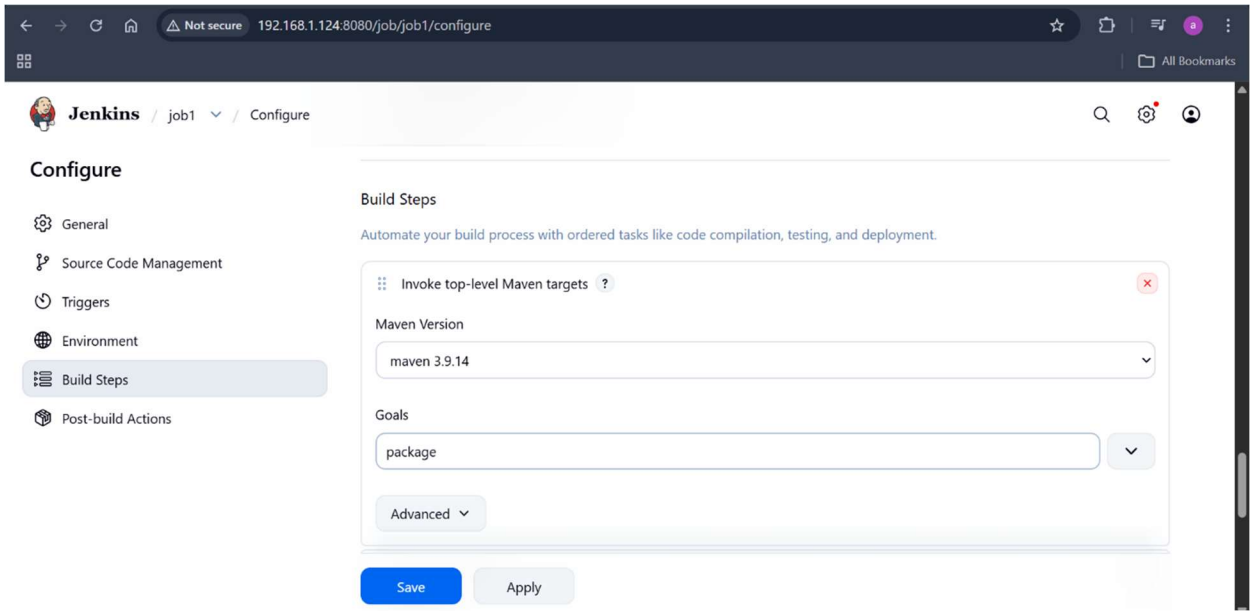

Not secure 192.168.1.124:8080/job/job1/configure
All Bookmarks
Jenkins / job1 / Configure
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Connect and manage your code repository to automatically pull the latest code for your builds.
None
Git
Repositories
Repository URL
https://github.com/ali1a1is/java-maven-app.git
Credentials
ali1a1is/****** (password based authentication for github)
Add
Advanced
Save
Apply

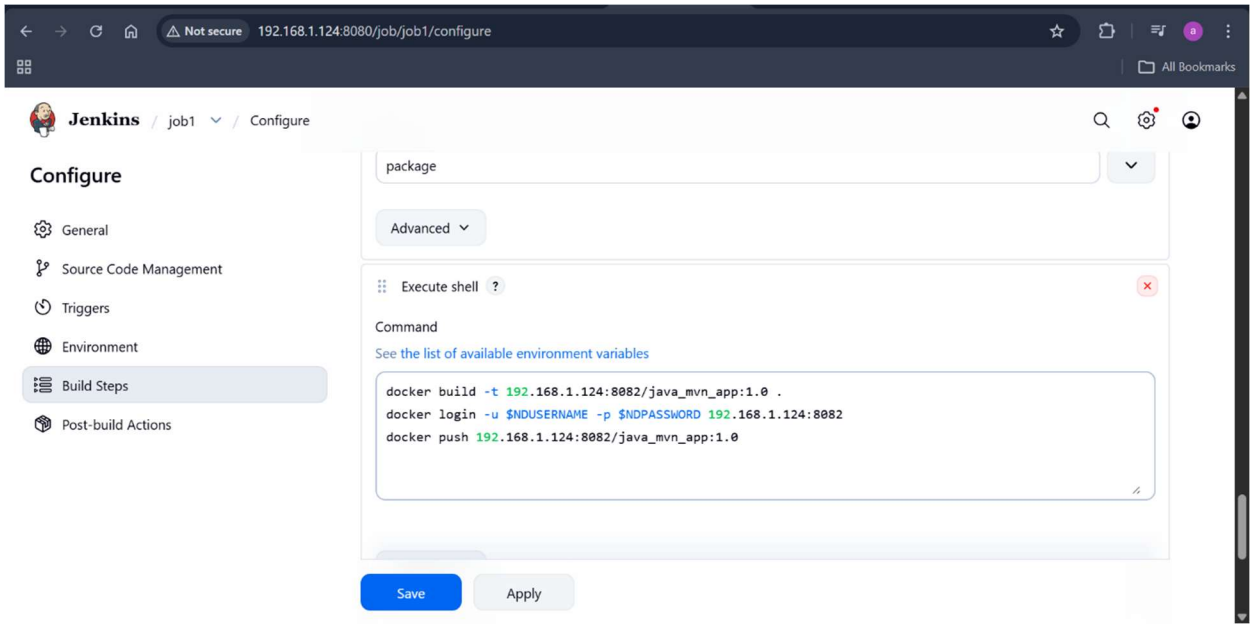

Not secure 192.168.1.124:8080/job/job1/configure
All Bookmarks
Jenkins / job1 / Configure
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Advanced
Add Repository
Branches to build
Branch Specifier (blank for 'any')
*/master
Add Branch
Repository browser
Save
Apply


Not secure 192.168.1.124:8080/job/job1/configure
All Bookmarks
Jenkins / job1 / Configure
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Environment
Configure settings and variables that define the context in which your build runs, like credentials, paths, and global parameters.
Delete workspace before build starts
Use secret text(s) or file(s)
Bindings
Username and password (separated)
Username Variable
DHRUSERNAME
Password Variable
DUHPASSWORD
Save
Apply

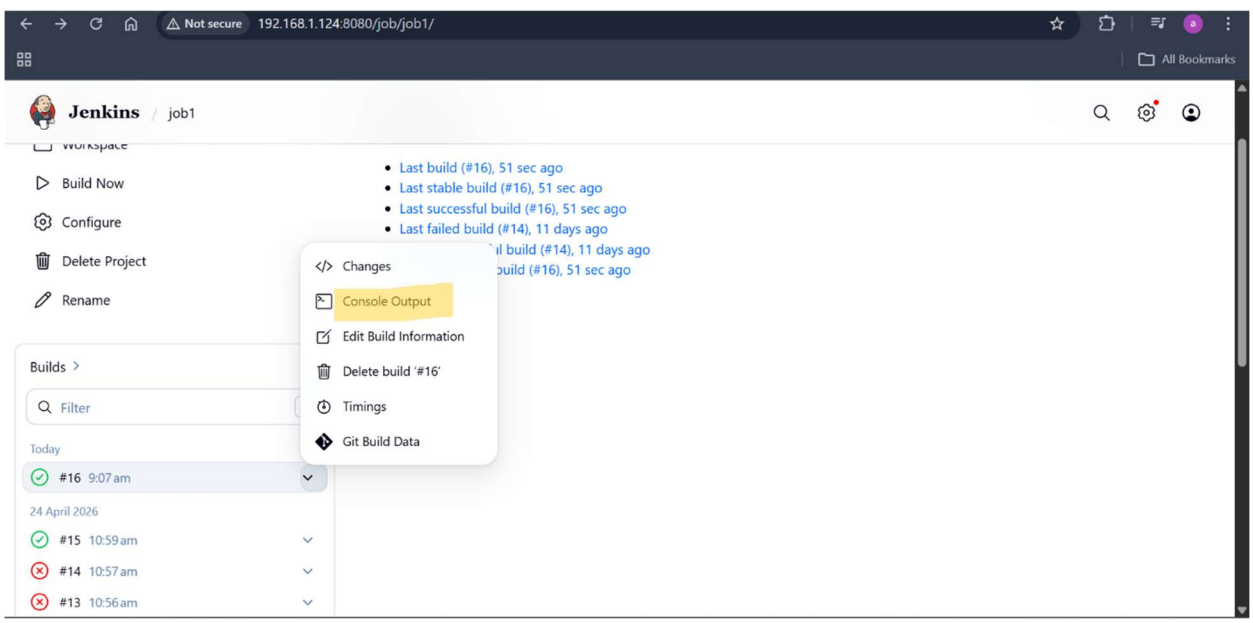

Not secure 192.168.1.124:8080/job/job1/configure
All Bookmarks
Jenkins / job1 / Configure
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
DHRUSERNAME
Password Variable
DUHPASSWORD
Credentials
Specific credentials
Parameter expression
ali2678/****** (username and password for dockerhub repo)
Add
Username and password (separated)
Username Variable
NDUSERNAME
Save
Apply

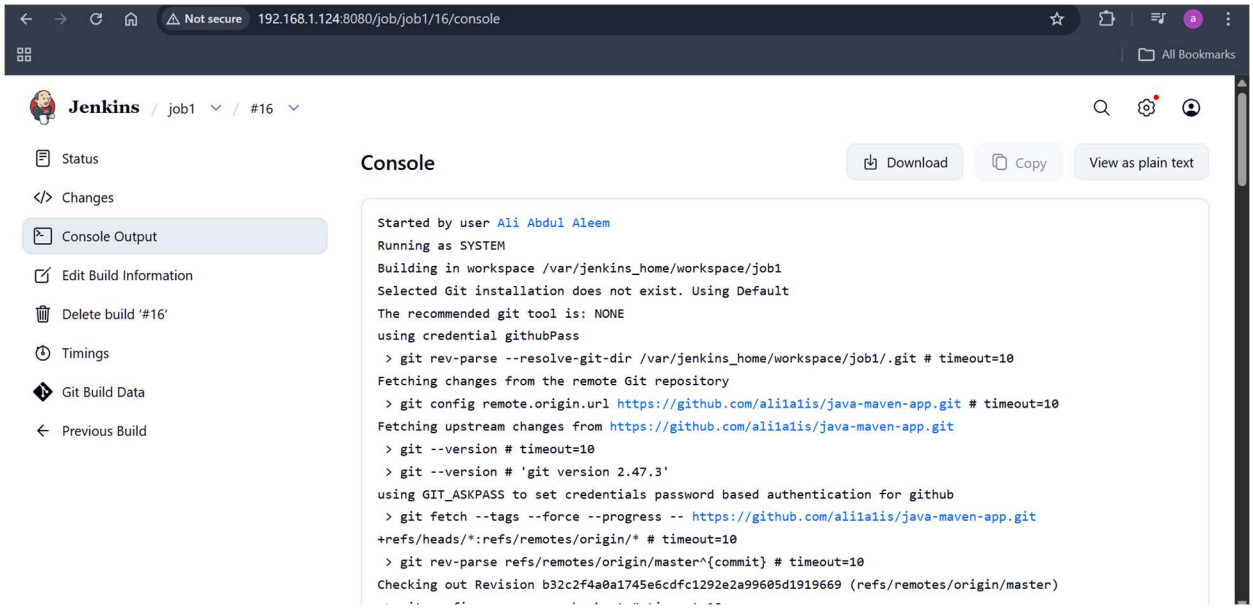

Not secure 192.168.1.124:8080/job/job1/configure
All Bookmarks
Jenkins / job1 / Configure
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Username Variable
NDUSERNAME
Password Variable
NDPASSWORD
Credentials
Specific credentials
Parameter expression
dockeruser/****** (docker repo user in nexus)
Add
Add
Add timestamps to the Console Output
Save
Apply

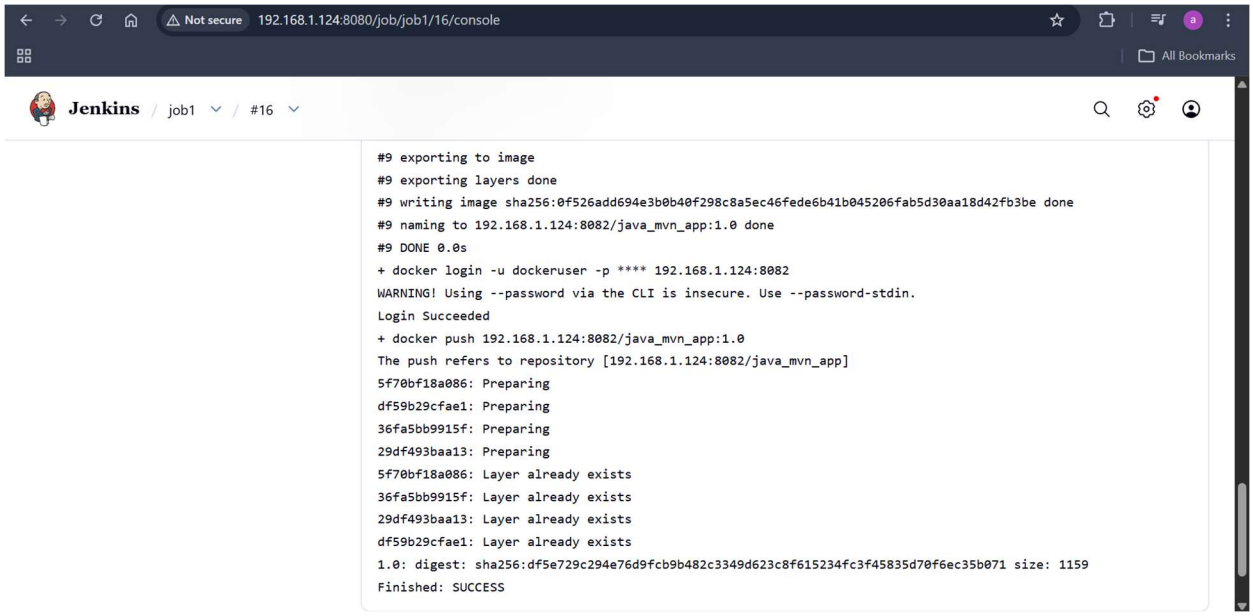

Not secure 192.168.1.124:8080/job/job1/configure
All Bookmarks
Jenkins / job1 / Configure
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Build Steps
Automate your build process with ordered tasks like code compilation, testing, and deployment.
Invoke top-level Maven targets
Maven Version
maven 3.9.14
Goals
package
Advanced
Save
Apply



Use the console output to view the execution details of the job:



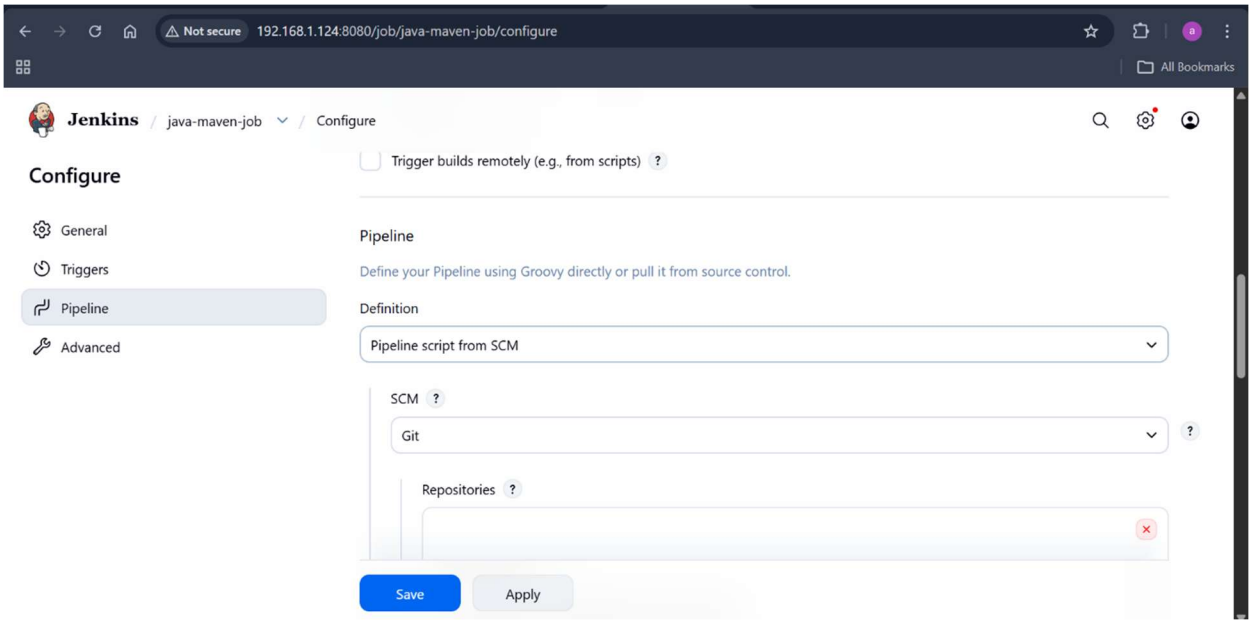


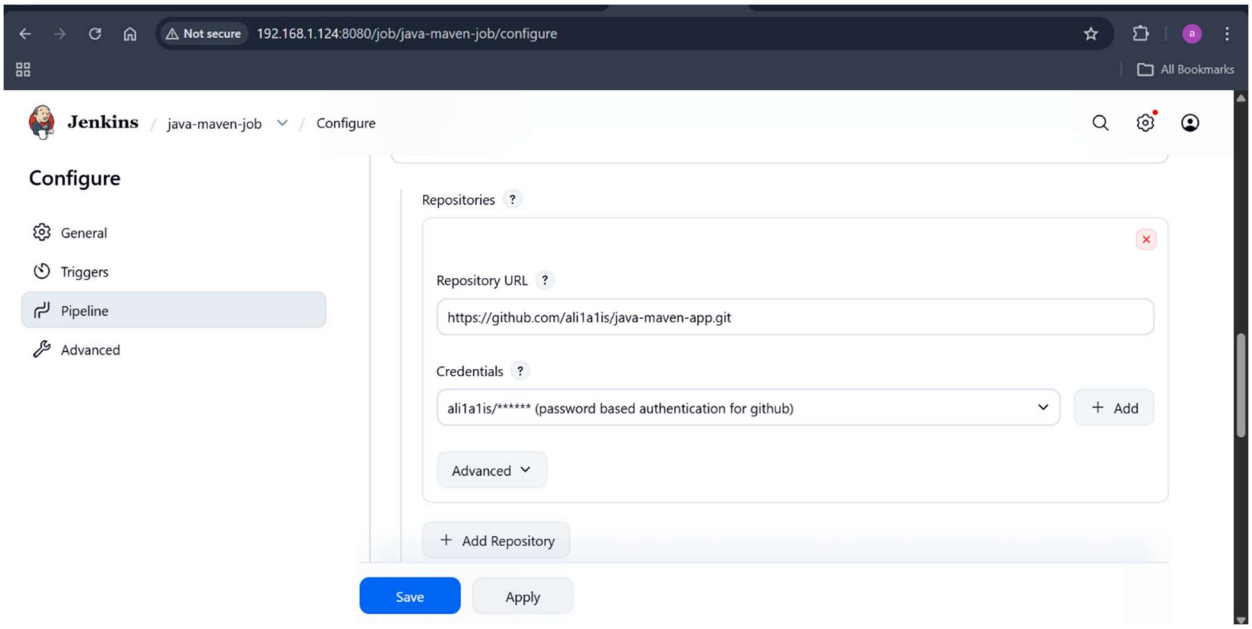


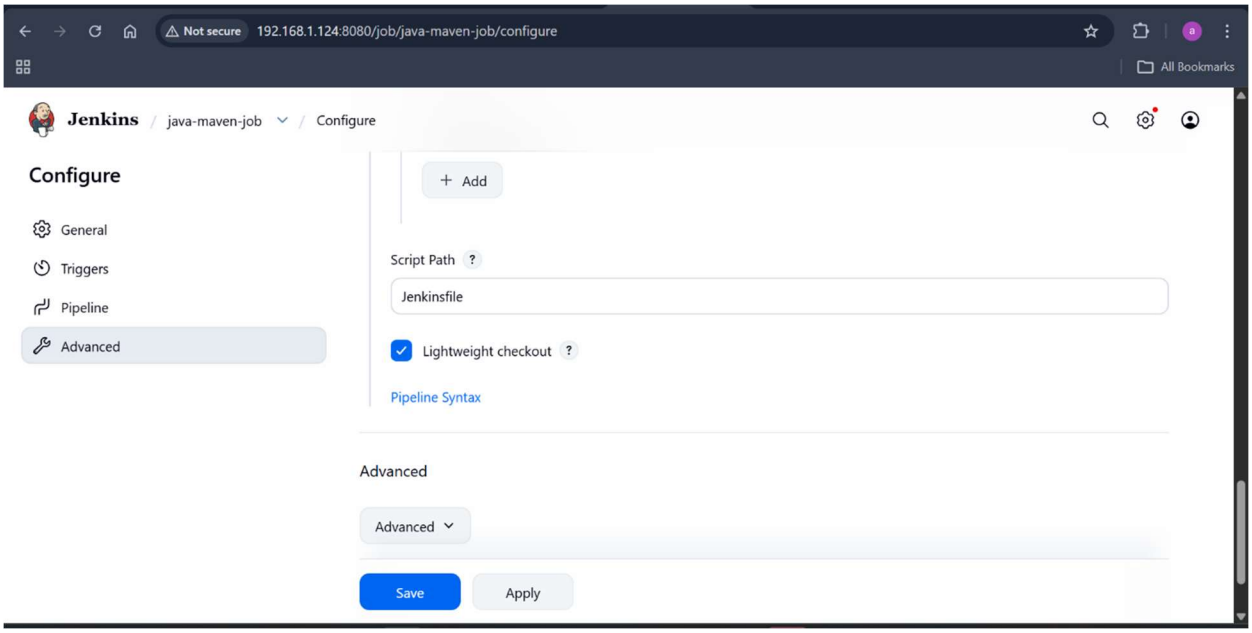
2. Pipeline Job

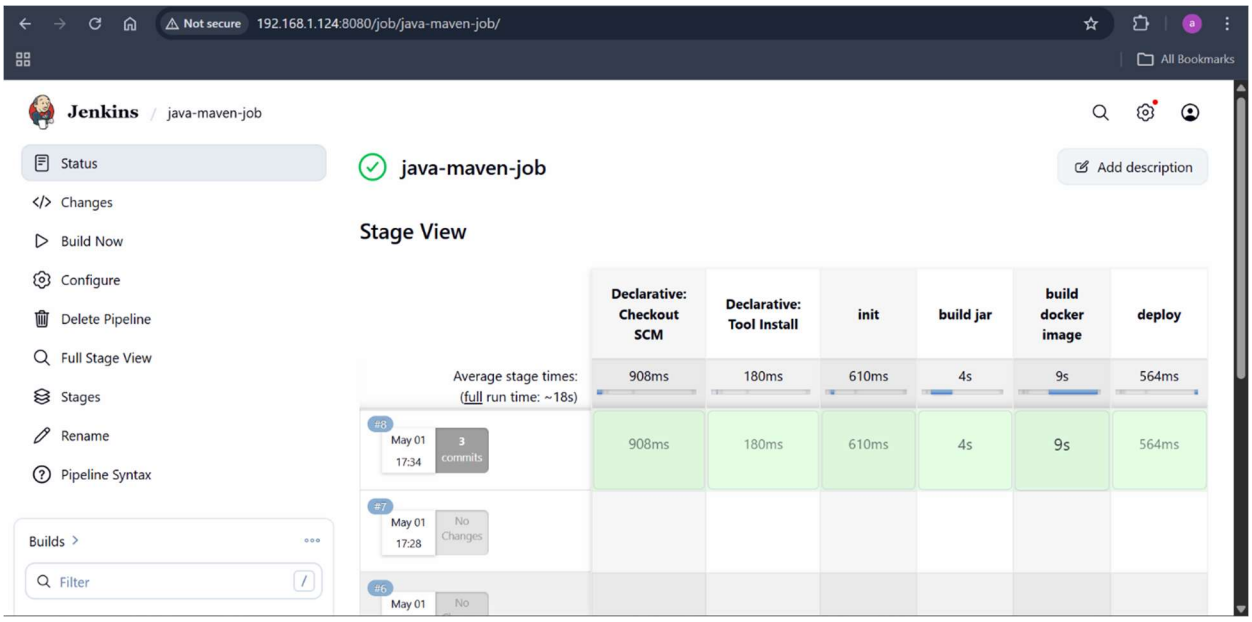
Uses a Jenkinsfile stored in the repository.

- Select the option to choose a pipeline script from a source code management (scm)
- Select git credentials
- Script path is Jenkinsfile


Not secure 192.168.1.124:8080/job/java-maven-job/configure
All Bookmarks
Jenkins / java-maven-job / Configure
Configure
General
Triggers
Pipeline
Advanced
Trigger builds remotely (e.g., from scripts)
Pipeline
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script from SCM
SCM
Git
Repositories
Save
Apply


Not secure 192.168.1.124:8080/job/java-maven-job/configure
All Bookmarks
Jenkins / java-maven-job / Configure
Configure
General
Triggers
Pipeline
Advanced
Repositories
Repository URL
https://github.com/ali1a1is/java-maven-app.git
Credentials
ali1a1is/****** (password based authentication for github)
Add
Advanced
Add Repository
Save
Apply





3**. Multibranch pipeline job**

Overview

The Multibranch Pipeline job was implemented in Jenkins to automate CI/CD workflows across multiple Git branches. Unlike traditional Pipeline jobs, the Multibranch Pipeline automatically detects branches from the connected Git repository and creates separate pipeline jobs for each branch.

In this project, two branches were configured:

- master
- feature

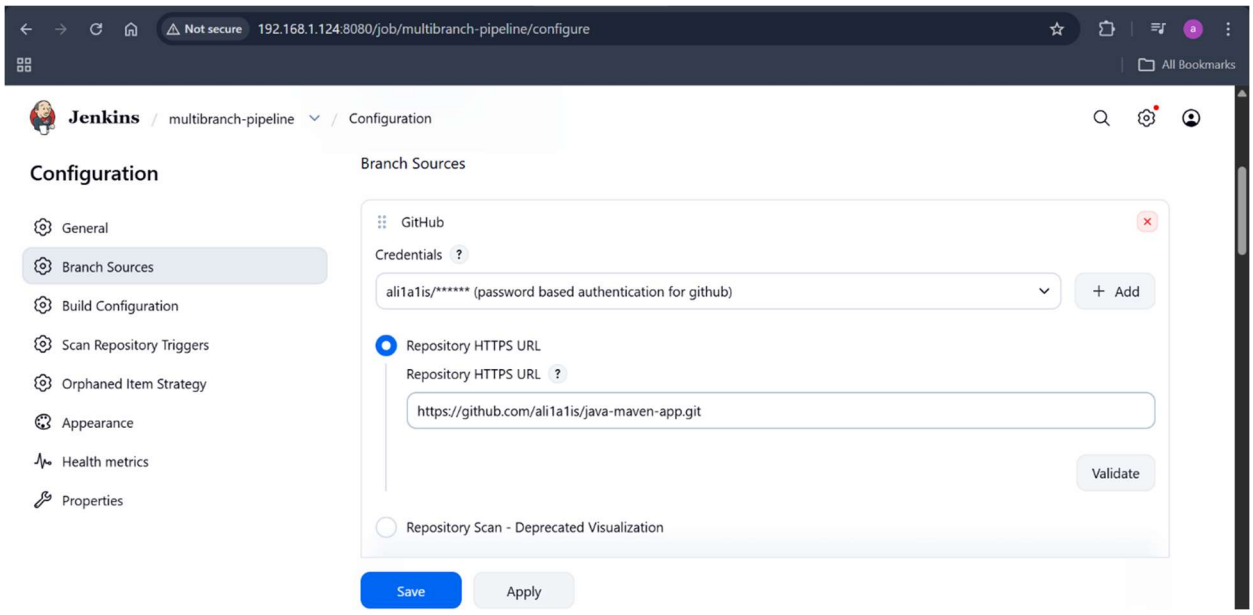
Each branch contains its own Jenkinsfile, allowing Jenkins to execute branch-specific workflows automatically whenever code changes are pushed to the repository.

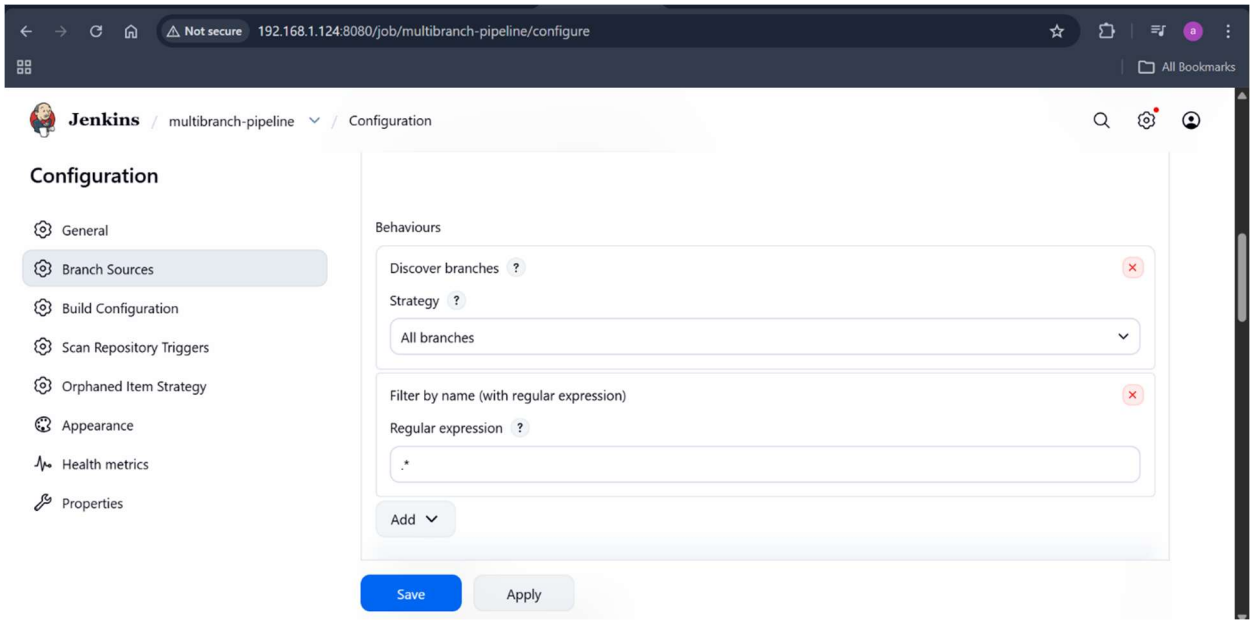
The pipeline was designed so that deployment-related stages only execute on the master branch, ensuring that development work in the feature branch does not affect the production environment.

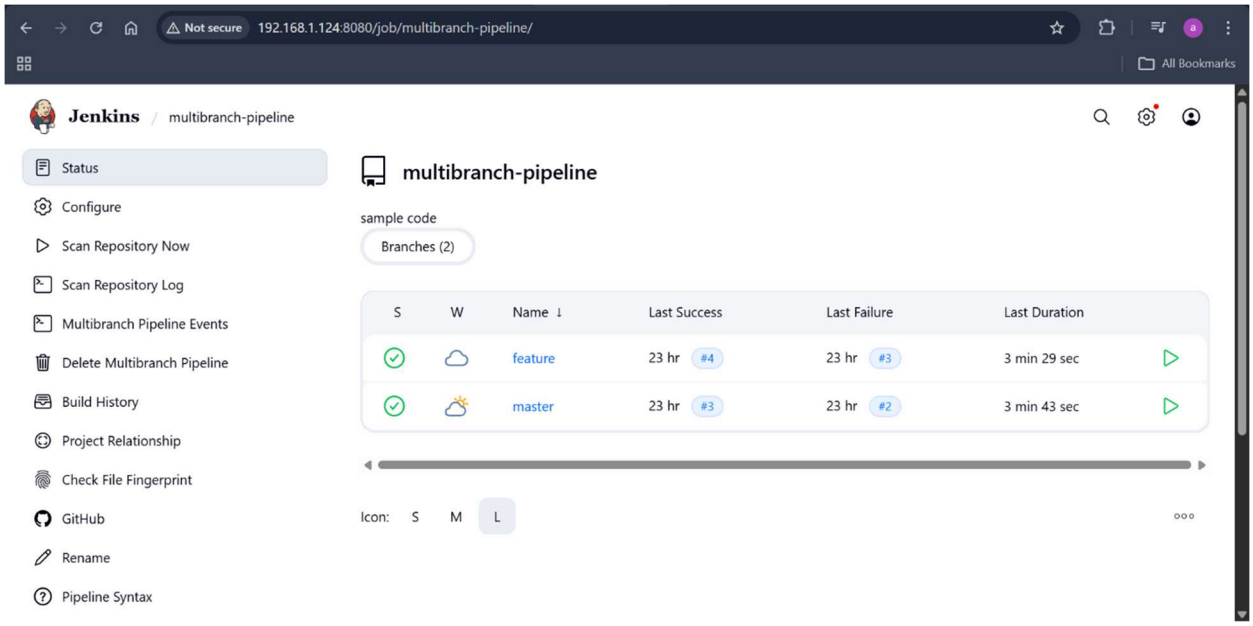
Objective

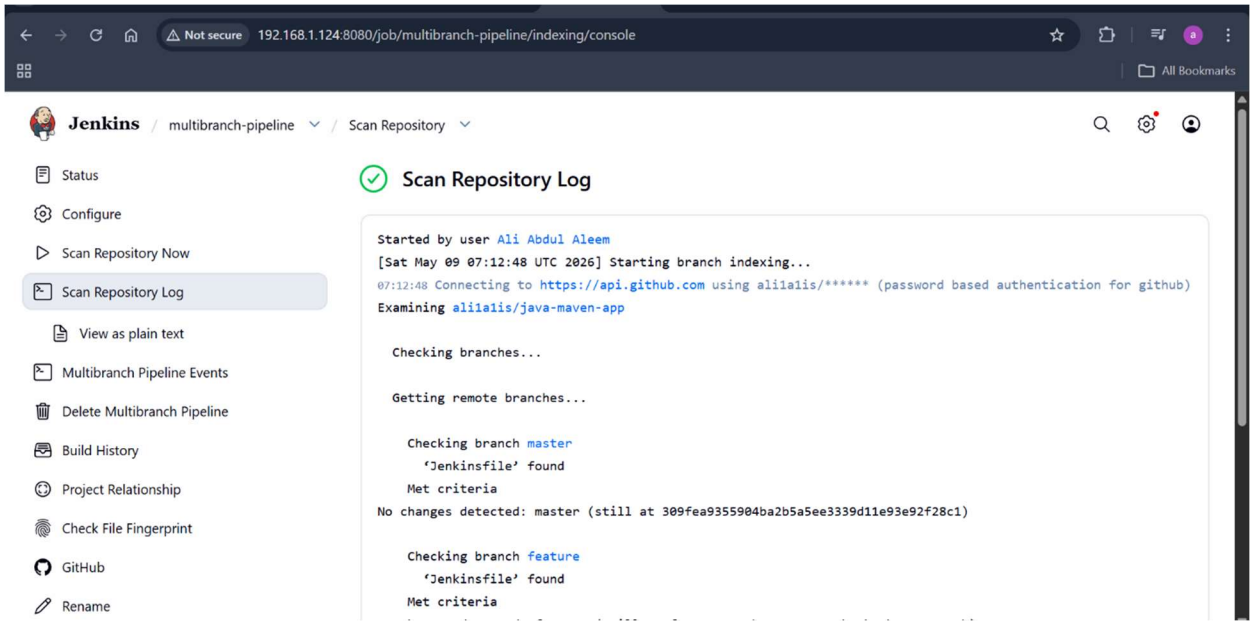
The objective of the Multibranch Pipeline implementation was to:

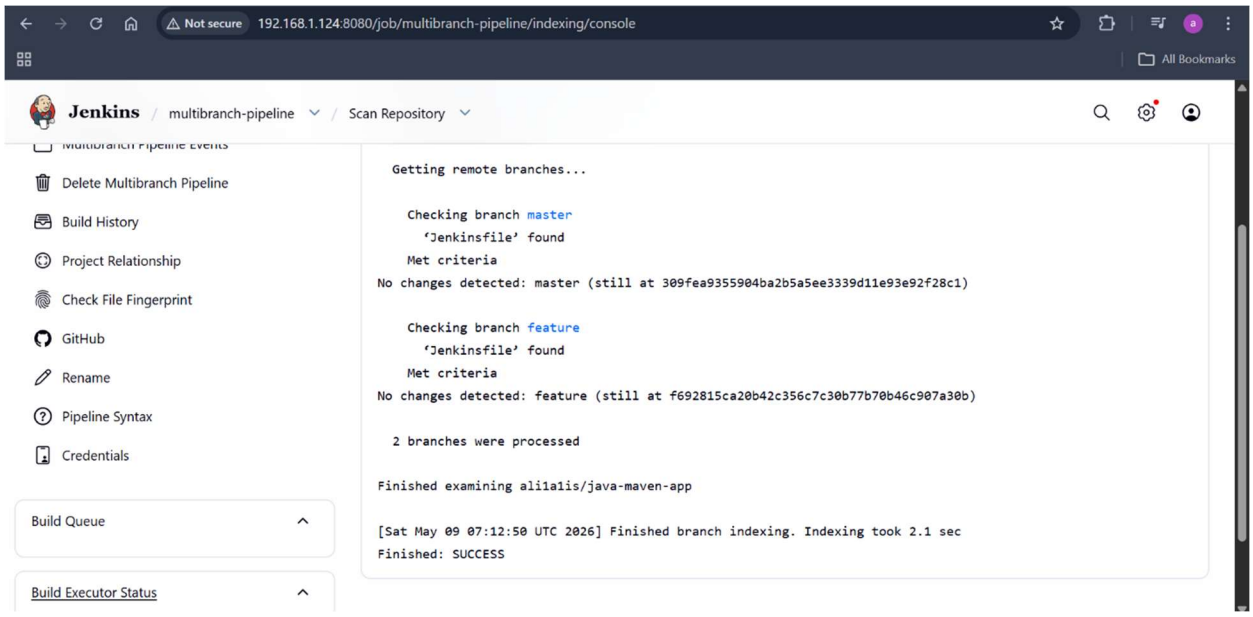
- Automate builds for multiple branches
- Separate development and production workflows
- Enable branch-specific CI/CD behaviour
- Improve scalability and maintainability of Jenkins pipelines
- Demonstrate advanced Jenkins Pipeline concepts




Not secure 192.168.1.124:8080/job/multibranch-pipeline/configure
All Bookmarks
Jenkins / multibranch-pipeline / Configuration
Configuration
General
Branch Sources
Build Configuration
Scan Repository Triggers
Orphaned Item Strategy
Appearance
Health metrics
Properties
Behaviours
Discover branches
Strategy
All branches
Filter by name (with regular expression)
Regular expression
.*
Add
Save
Apply


Not secure 192.168.1.124:8080/job/multibranch-pipeline/
All Bookmarks
Jenkins / multibranch-pipeline
Status
Configure
Scan Repository Now
Scan Repository Log
Multibranch Pipeline Events
Delete Multibranch Pipeline
Build History
Project Relationship
Check File Fingerprint
GitHub
Rename
Pipeline Syntax
multibranch-pipeline
sample code
Branches (2)
S W Name ↓ Last Success Last Failure Last Duration
feature 23 hr #4 23 hr #3 3 min 29 sec
master 23 hr #3 23 hr #2 3 min 43 sec
Icon: S M L





Jenkinsfile Structure

The pipeline uses the Declarative Pipeline syntax.

Global Variable Loading

def gv

A global variable named gv is declared. This variable is later used to load reusable Groovy functions from the script.groovy file.

Pipeline Configuration

```
pipeline {
   agent any
```

Agent Configuration

The pipeline can run on any available Jenkins agent.

Maven Tool Configuration

```
tools{
   maven "maven 3.9.14"
}
```

The pipeline uses Maven version 3.9.14 configured in Jenkins Global Tool Configuration.

This allows Jenkins to automatically provide Maven during execution without manual installation inside the pipeline.

Pipeline Stages

1. Init Stage

```
stage("init")
```

Purpose

The init stage loads external reusable Groovy scripts.

Implementation

```
gv = load("script.groovy")
```

The script.groovy file contains reusable functions such as:

- buildJar()
- buildImage()
- deployApp()

This approach improves modularity and keeps the Jenkinsfile clean and maintainable.

2. Build Jar Stage

```
stage("build jar")
```

Purpose

Compiles the Java Maven application and generates the JAR artifact.

Branch Restriction

BRANCH_NAME == "master"

This stage only executes when the pipeline is running from the master branch.

Function Called

gv.buildJar()

The function performs the Maven build process, typically using commands such as:

mvn package

Outcome

- Generates the application JAR file
- Verifies that the application compiles successfully

3. Build Docker Image Stage

stage("build docker image")

Purpose

Builds a Docker image for the application.

Branch Restriction

The stage only runs on the master branch.

Function Called

gv.buildImage()

Outcome

- Creates a Docker image from the application
- Prepares the application for containerized deployment

This stage demonstrates Jenkins integration with Docker.

4. Deploy Stage

stage("deploy")

Purpose

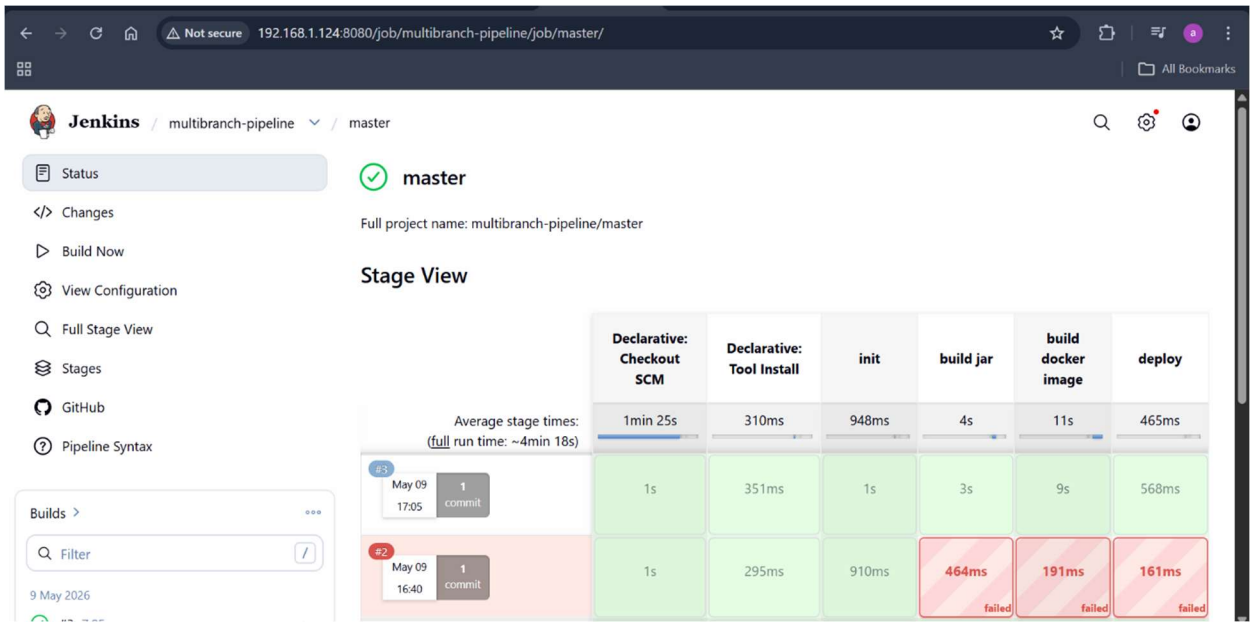
Deploys the application after successful build and image creation.

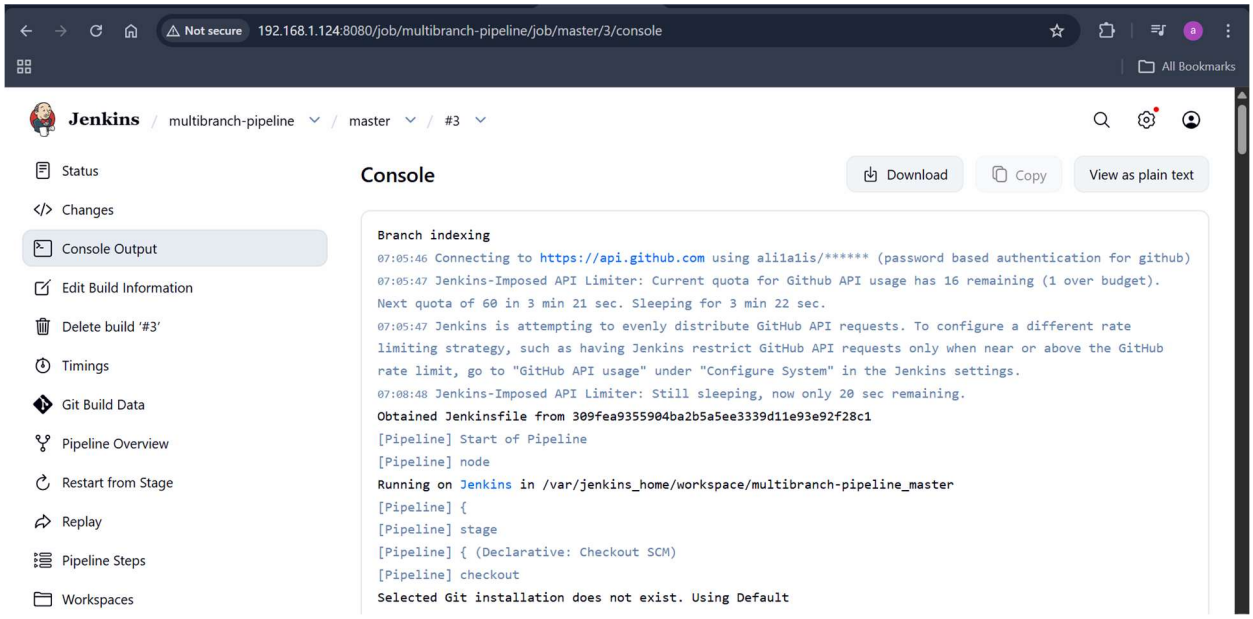
Branch Restriction

This stage only runs on the master branch.

Function Called

gv.deployApp()





Not secure 192.168.1.124:8080/job/multibranch-pipeline/job/master/3/console

All Bookmarks

Jenkins / multibranch-pipeline / master / #3

```
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline

Could not update commit status, please check if your scan credentials belong to a member of the organization
or a collaborator of the repository and repo:status scope is selected


GitHub has been notified of this commit’s build result

Finished: SUCCESS
```

Jenkins 2.541.3

Not secure 192.168.1.124:8080/job/multibranch-pipeline/job/feature/

All Bookmarks

Jenkins / multibranch-pipeline / feature

- Status
- Changes
- Build Now
- View Configuration
- Full Stage View
- Stages
- GitHub
- Pipeline Syntax

Builds >

Filter /

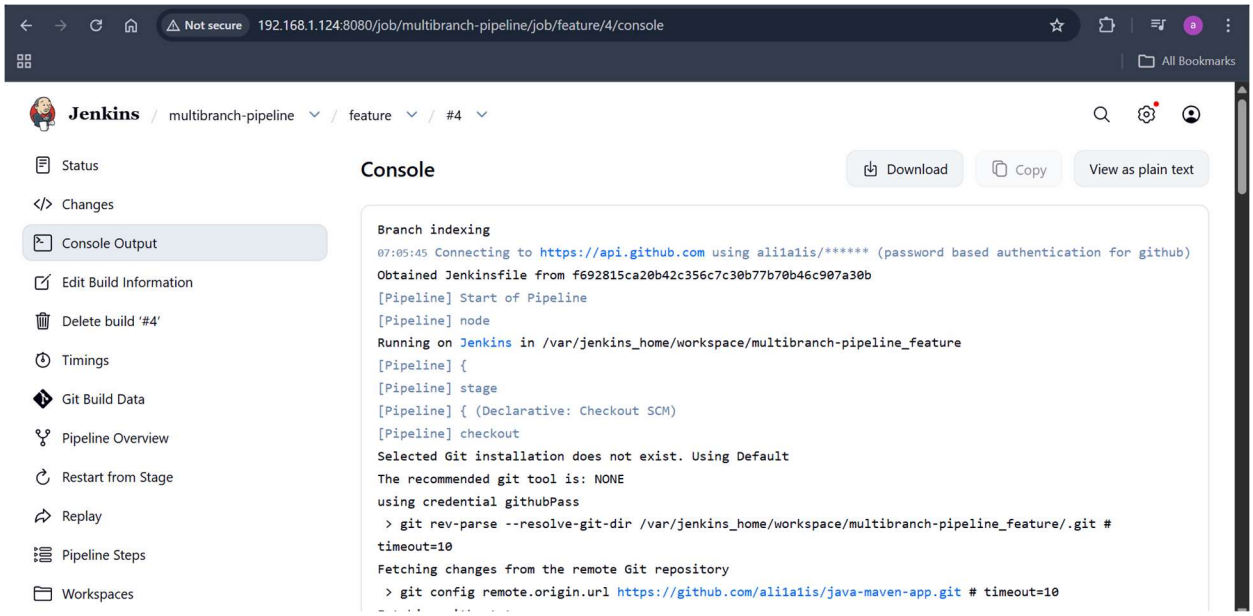
9 May 2026

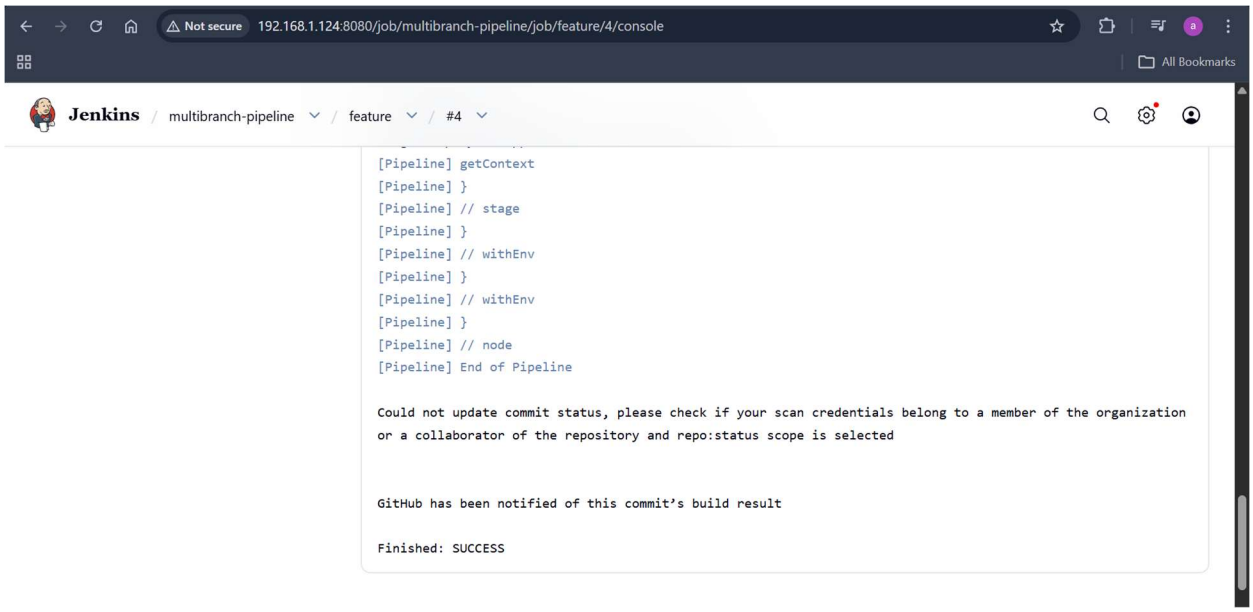
feature

Full project name: multibranch-pipeline/feature

Stage View

	Declarative: Checkout SCM	Declarative: Tool Install	init	testing	build jar	build docker image	deploy
Average stage times: (full run time: ~3min 29s)	4min 42s	239ms	830ms	717ms	470ms	183ms	198ms
#4 May 09 17:05 1 commit	3min 23s	198ms	697ms	873ms			
#3 May 09 16:36 1 commit	6min 7s	256ms	909ms	624ms	433ms failed	227ms failed	221ms failed





Outcome

- Deploys the application automatically
- Simulates a Continuous Deployment workflow

The feature branch is primarily used for development and testing, while the master branch represents the stable production-ready branch.

This separation ensures safer deployments and follows standard Git branching practices.

Key Jenkins Concepts Demonstrated

Multibranch Pipeline

Automatically scans the Git repository and creates pipelines for each branch.

Declarative Pipeline

Uses structured Jenkins Pipeline syntax for readability and maintainability.

Shared Script Loading

Reusable Groovy functions are loaded externally using:

load("script.groovy")

Conditional Execution

Branch-based execution is implemented using:

BRANCH_NAME == "master"

CI/CD Automation

The pipeline automates:

- Build
- Packaging
- Docker image creation
- Deployment

Benefits of the Multibranch Pipeline

- Reduced manual configuration
- Better branch isolation
- Easier management of development workflows
- Automated CI/CD for multiple branches
- Improved scalability for larger projects

Conclusion

The Multibranch Pipeline implementation successfully demonstrated advanced Jenkins automation capabilities using branch-aware CI/CD workflows. By integrating Maven, Docker, Groovy scripting, and conditional stage execution, the project achieved a clean and scalable pipeline architecture.

The use of separate feature and master branches ensured controlled deployments while supporting parallel development workflows, reflecting real-world DevOps and Continuous Integration practices.